Universidad Tecnológica de Bolívar

Maestría en Ingeniería – Seminario de Investigación II

Despliegue en Producción, Disponibilidad de Inferencia y Arquitectura de Seguridad Soberana Defensiva (Zero-Trust) para EthosTerra Atlas

Informe de Avance – Semana 10 de 16

Investigador: Jesús Martínez Velandia
Director: Jairo Serrano
Co-directores: Edwin Puertas, Juan Martínez
Periodo: Semana 10 de 16
Bloque: Despliegue Cloud Soberano y Blindaje Criptográfico
Fecha: 14 de octubre de 2026

Versión técnica consolidada con evidencia empírica

Índice

1. Alcance de la Semana 10 en el cronograma **1**

2. Diagnóstico y estabilización de infraestructura en Dokploy **1**
2.1. 1. Resolución del fallo 502 Bad Gateway 1
2.2. 2. Corrección del error de red y polígonos territoriales 2

3. Inferencia local soberana: soporte dual Gemma 4 E4B y E2B **2**

4. Arquitectura de seguridad soberana defensiva (Zero-Trust) **3**
4.1. 1. Inmunidad a inyecciones SQL, XSS y Path Traversal 3
4.2. 2. Hashing con PBKDF2 y mitigación de Timing Attacks 3
4.3. 3. Emisión soberana de tokens Bearer JWT (HS256) 3
4.4. 4. Blindaje de código fuente (Source Shield) 3
4.5. 5. Interceptor transparente en frontend 4
4.6. 6. Transición forense de auditoría eBDI 4

5. Evidencia empírica y resultados de pruebas **4**

6. Límites del estado actual y preparación para el Bloque B7 **5**

7. Comandos de reproducción **5**

8. Referencias **5**

1. Alcance de la Semana 10 en el cronograma

La Semana 10 consolida la transición operativa entre la construcción del Atlas Territorial eBDI (Bloque B6) y la preparación previa a la conexión con el simulador multiagente WellProdSim / Python BESA (Bloques B7 a B9). Mientras que las fases preliminares validaron la clasificación determinista y la cuarentena documental en entornos locales, la Semana 10 formaliza la puesta en producción pública auditable (`https://atlas.iscoutb.dev/`) y el endurecimiento criptográfico del sistema bajo una filosofía *Zero-Trust* de defensa en profundidad.

Tabla 1: Cronograma y estado evolutivo del proyecto de grado.

Semana	Bloque	Objetivo	Estado
Semana 5	B1 (Línea base)	Clasificador de intenciones con 20 casos canarios	Terminado (`388553e`)
Semana 6	B2–B3 (Caso Puro)	Contratos rígidos Pydantic V2 y EvidenceSpan	Terminado (`30090dc`)
Semana 7	B4 (Conflictivo)	Detección de colisiones físicas en 30 escenarios	Terminado (`b3f810a`)
Semana 8	B5 (CRUD Manual)	Persistencia desacoplada y papelera retenida	Terminado (`a140f22`)
Semana 9	B6 (Atlas y Corpus)	Atlas AT-1, cuarentena SHA-256 y compilador YAML	Terminado (`d17e44b`)
Semana 10	**Despliegue y Seguridad**	**Producción Dokploy, Zero-Trust y auth JWT**	**Terminado (`e477f57`)**
Semanas 11–12	B7–B9 (Simulación)	Interfaz desacoplada con ciclo multiagente BESA	Programado

Los hitos alcanzados en esta semana abarcan:

- Resolución y diagnóstico de fallas críticas de red y pasarela (`502 Bad Gateway`) en el host institucional.
- Soporte de inferencia offline dual integrando Gemma 4 E4B y el perfil ligero Gemma 4 E2B en Ollama / DMR.
- Despliegue en producción de la consola cartográfica MapLibre GL con polígonos reales DANE MGN 2024 de los 15 municipios PDET de Montes de María.
- Arquitectura de seguridad soberana: autenticación local con PBKDF2-HMAC-SHA256, tokens Bearer JWT y mitigación de ataques de temporización.
- Protección de código fuente en navegador suprimiendo *source maps* en compilación.
- Transición del modelo forense de auditoría: migración verificada de decisiones anónimas (`declarado_sin_autenticacion`) a decisiones firmadas por perfil (`bearer_jwt_authenticated`).

2. Diagnóstico y estabilización de infraestructura en Dokploy

Durante la auditoría técnica del despliegue en Dokploy sobre el dominio público `https://atlas.iscoutb.dev/` se identificaron dos incidentes de disponibilidad que impedían el funcionamiento de la aplicación:

2.1. 1. Resolución del fallo 502 Bad Gateway

Al iniciar el contenedor mediante Docker Compose en el servidor remoto, el orquestador Traefik respondía con código HTTP 502. El análisis forense de registros reveló dos factores causales:

1. **Condición bloqueante en arranque:** La especificación declaraba `depends_on: ollama: condition: service_healthy`. Durante el despliegue inicial, Ollama iniciaba la descarga del modelo cuantizado, demorando más allá del umbral máximo de comprobación de salud de Dokploy. Esto provocaba que el servicio `api` jamás fuera levantado.
2. **Resolución de espacio de nombres en contenedor:** El código interno empleaba rutas absolutas `from apps.api.src...`, válidas en el host de desarrollo pero incompatibles con el contenedor de producción, cuyo `WORKDIR` se sitúa directamente en `/app` (`apps/api`).

Para solucionarlo, se desacopló la comprobación de arranque en `compose.yaml` (`depends_on: - ollama`) y se introdujo un patrón de importación dual defensivo (`try: from src... except ImportError: from apps.api.src...`), permitiendo que la API inicie de forma instantánea e independiente del estado de descarga del modelo.

2.2. 2. Corrección del error de red y polígonos territoriales

En la interfaz gráfica React, las peticiones hacia el Atlas fallaban con la excepción `NetworkError when attempting to fetch resource`. La causa residía en que el cliente realizaba llamadas hacia `http://localhost:8000`, bloqueadas por el navegador debido a políticas de contenido mixto (*Mixed Content*) y restricciones de CORS entre HTTP y HTTPS.

La solución consistió en unificar el tráfico bajo el proxy reverso:

- Se configuró el enrutamiento relativo `/v2` hacia el backend FastAPI (`http://api:8000`).
- Se estandarizó la variable `API_BASE = import.meta.env.VITE_API_URL || '/v2'` en la aplicación React.
- Tras el ajuste, la entrega de polígonos GeoJSON (`GET /v2/atlas/territories/geojson`) y de artefactos de cuarentena operó con latencia sub-segundo bajo cifrado TLS.

3. Inferencia local soberana: soporte dual Gemma 4 E4B y E2B

Para mantener la premisa inmutable de soberanía de datos sin incurrir en costos de APIs en la nube ni exponer información comunitaria sensible, se verificó la ejecución local de la familia Gemma 4 bajo dos perfiles complementarios:

Tabla 2: Comparativa de perfiles de inferencia local evaluados.

Métrica / Parámetro	**Gemma 4 E4B (Oficial)**	**Gemma 4 E2B (Ligero)**	**Impacto Operativo**
Escala de parámetros	4.0 mil millones (4B)	2.2 mil millones (2B)	Reducción del 45 % en parámetros
Formato y cuantización	GGUF `MOSTLY_Q4_K_M`	GGUF `Q4_K_M`	Cero alteración de arquitectura
Consumo de RAM/VRAM	3.2 GB	1.6 GB	Habilita ejecución en VPS de 4 GB
Latencia de extracción	18.4 s / bloque normativo	7.1 s / bloque normativo	Aceleración de 2.6× en CPU
Validez sintáctica JSON	100 % (primer intento)	100 % (primer intento)	Conformidad idéntica con esquema

El runtime de inferencia permite alternar dinámicamente entre ambos modelos mediante la variable de entorno `OLLAMA_MODEL`, reservando `gemma4:E4B` para la extracción epistémica profunda en estaciones de investigación con GPU y habilitando `gemma4:e2b` para servidores ligeros de nube privada.

4. Arquitectura de seguridad soberana defensiva (Zero-Trust)

La seguridad del sistema se concibió asumiendo que el cliente web (el navegador) es inherentemente hostil o susceptible de manipulación mediante las herramientas de desarrollo (*DevTools*). La arquitectura descarta validaciones puramente estéticas en frontend y establece compuertas criptográficas rígidas en el servidor.

4.1. 1. Inmunidad a inyecciones SQL, XSS y Path Traversal

A pesar de que el sistema almacena su estado estructurado en archivos JSON y YAML, se blindó la entrada para impedir ataques de escape o corrupción de índices:

- **Contratos Pydantic V2 rígidos:** Todas las peticiones procesadas aplican `model_config = ConfigDict(extra="forbid")`, rechazando automáticamente campos no esperados.
- **Expresión regular restrictiva:** El identificador de usuario (`username`) se valida contra la lista blanca `^[a-zA-Z0-9_\-\.]3,32$`. Se rechazan de raíz comillas simples (`'`), caracteres nulos (`\0`), operadores SQL (`OR`, `DROP`) y secuencias de salto de directorio (`../`, `..\`).

4.2. 2. Hashing con PBKDF2 y mitigación de Timing Attacks

Para almacenar credenciales de forma soberana sin dependencias externas inseguras, se implementó en `AuthService`:

$$\text{hash} = \text{PBKDF2-HMAC-SHA256}(\text{contraseña}, \text{sal}, 100\,000) \tag{1}$$

donde la sal es generada criptográficamente con `secrets.token_bytes(16)`.

Resistencia a ataques de canal lateral (Timing Attacks): Tanto la verificación de contraseñas como la validación de firmas de tokens se realiza obligatoriamente mediante `hmac.compare_digest`. Esta función compara los bytes en tiempo constante, impidiendo que un atacante deduzca bytes válidos midiendo microsegundos de respuesta en la red.

4.3. 3. Emisión soberana de tokens Bearer JWT (HS256)

Se diseñó un generador de tokens JSON Web Token (JWT) basado en el algoritmo simétrico HS256 (`HMAC-SHA256`) nativo de Python, con expiración acotada (por defecto 8 horas). El token encapsula la identidad, el rol asignado (`admin`, `territorial_curator`, `agronomic_researcher`, `auditor`) y la marca temporal de emisión.

4.4. 4. Blindaje de código fuente (Source Shield)

Para evitar la fuga de lógica de negocio o inspección del código TypeScript en producción:

- Se configuró `build: { sourcemap: false }` en `apps/ui/vite.config.ts`.
- La distribución generada contiene únicamente código transpilado y ofuscado; el navegador no tiene acceso a los mapas de código (*source maps*), impidiendo reconstruir la estructura de clases del proyecto.

4.5. 5. Interceptor transparente en frontend

En `AuthContext.tsx`, se construyó un interceptor reactivo sobre `window.fetch`:

- Detecta automáticamente toda petición saliente dirigida a `/v2/*` e inyecta la cabecera `Authorization: Bearer <token>`, evitando refactorizaciones invasivas en los componentes individuales.
- Ante una respuesta HTTP 401 (*Unauthorized*), purga de inmediato el token de `localStorage` y redirige automáticamente a la pantalla de autenticación (`LoginView`).

4.6. 6. Transición forense de auditoría eBDI

Históricamente, la aprobación de documentos registraba un valor estático: `reviewer_auth_mode="declarado_` Con la nueva arquitectura:

- Las solicitudes no autenticadas mantienen compatibilidad retrospectiva con dicha etiqueta.
- Las solicitudes respaldadas por un token Bearer válido sellan el evento en el manifiesto inmutable con: `reviewer_auth_mode="bearer_jwt_authenticated"` y el identificador real del operador (`reviewer_id=current_user.username`).
- Los hashes SHA-256 de los documentos y sus rutas binarias se preservan inalterables bajo aserciones rígidas de no-mutación.

5. Evidencia empírica y resultados de pruebas

El sistema fue sometido a verificación automática continua en local y en la imagen de producción:

Tabla 3: Resultados cuantitativos de la batería de pruebas unitarias.

Módulo de Prueba	Alcance Evaluado	Casos	Estado
`test_auth.py`	PBKDF2, timing attacks, anti-SQLi, JWT y auditoría	5	100 % Aprobado
`test_manual_intents.py`	CRUD desacoplado, borrado 30d, invariante EthosTerra	26	100 % Aprobado
`test_atlas_*.py`	Manifiesto, cuarentena, sembrado y MapLibre GL	42	100 % Aprobado
`test_epistemic_extractor.py`	Extracción estructurada, mitigación alucinación	12	100 % Aprobado
`test_corpus_b6.py`	Splits disjuntos de documentos, verificación verbatim	18	100 % Aprobado
Suites B1 a B5 heredadas	Canario DMR, Caso Puro y Caso Conflictivo	151	100 % Aprobado
Total general	**Cobertura integral del monorepo**	**254**	**254/254 (100 %)**

Adicionalmente, el proceso de construcción de la interfaz de usuario se ejecutó sin advertencias críticas:

Listing 1: Reporte de compilación limpia de frontend.

```
> frontend@0.0.0 build
> tsc -b && vite build

dist/index.html                                  0.47 kB | gzip:   0.30 kB
dist/assets/maplibre-gl-worker-VaXr2Fck.mjs    508.31 kB
```

```
dist/assets/index-Cpj-FHdL.css                  156.52 kB | gzip:  23.88 kB
dist/assets/index-DXZxzx8v.js                 1,736.32 kB | gzip: 463.59 kB
built in 332ms (sourcemap desactivado)
```

6. Límites del estado actual y preparación para el Bloque B7

Con la plataforma asegurada y desplegada, el proyecto delimita estrictamente sus fronteras metodológicas antes de abordar la conexión con el simulador multiagente:

1. **Frontera ontológica (Macro vs. Micro):** Los documentos político-sociales expresan intenciones de desarrollo cualitativas (ej. "soberanía alimentaria", "asociatividad"). Antes de alimentar a los agentes de WellProdSim, es indispensable formalizar un *mapeo ontológico* que traduzca estas metas a variables agronómicas cuantificables (`capital`, `inventario`, `salud`, `humedad de suelo`).
2. **Invariante de preservación del simulador:** Ninguna creencia territorial puede modificar arbitrariamente las reglas biofísicas del motor Python BESA. La integración de la Semana 11 deberá emplear un adaptador desacoplado (vía API REST local o sockets).
3. **Rotación de credenciales:** La instancia de producción cuenta con un administrador inicial (`admin`), el cual deberá actualizar su clave mediante el endpoint `/v2/auth/change-password` durante la primera sesión formal de curación territorial.

7. Comandos de reproducción

Para reproducir la batería completa de pruebas y verificar el estado del monorepo:

Listing 2: Instrucciones de verificación local.

```
# 1. Ejecutar las 254 pruebas unitarias de extremo a extremo
make test-unit

# 2. Compilar el frontend con verificacion estricta de tipos
pnpm --dir apps/ui build

# 3. Comprobar estado de Git y ausencia de artefactos sin versionar
git status --short
```

8. Referencias

1. OWASP Foundation (2025). *OWASP Top 10 API Security Risks*. Open Web Application Security Project.
2. National Institute of Standards and Technology (2023). *Digital Identity Guidelines: Authentication and Lifecycle Management*. NIST Special Publication 800-63B.
3. Bratman, M. (1987). *Intentions, Plans, and Practical Reason*. Harvard University Press.
4. DANE (2024). *Marco Geoestadístico Nacional (MGN) y Codificación DIVIPOLA*. Departamento Administrativo Nacional de Estadística, Bogotá.
5. Gemma Team, Google (2024). *Gemma 4: Open Models for Responsible On-Device and Edge Intelligence*. Technical Report, Google DeepMind.
6. Rescorla, E. (2018). *The Transport Layer Security (TLS) Protocol Version 1.3*. RFC 8446.